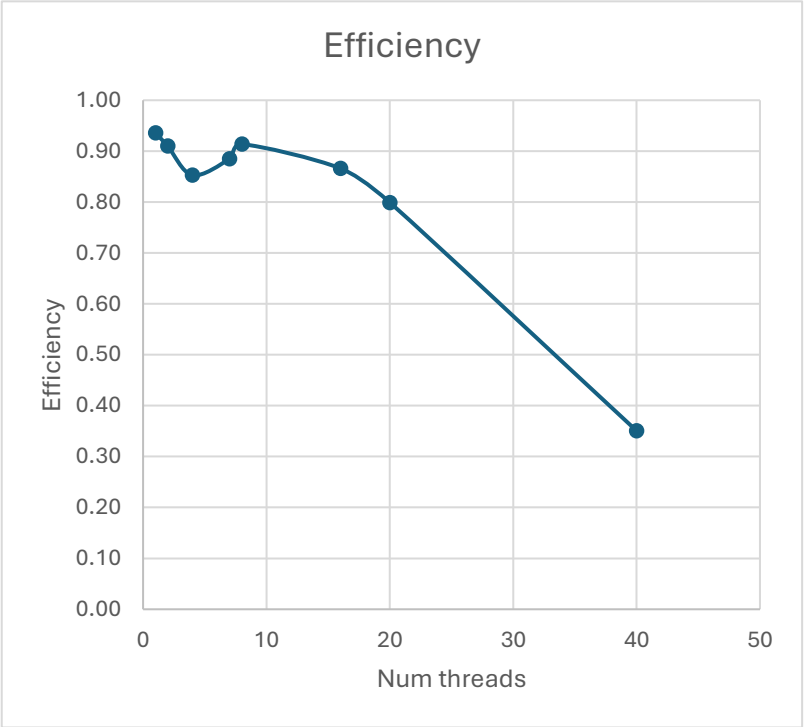
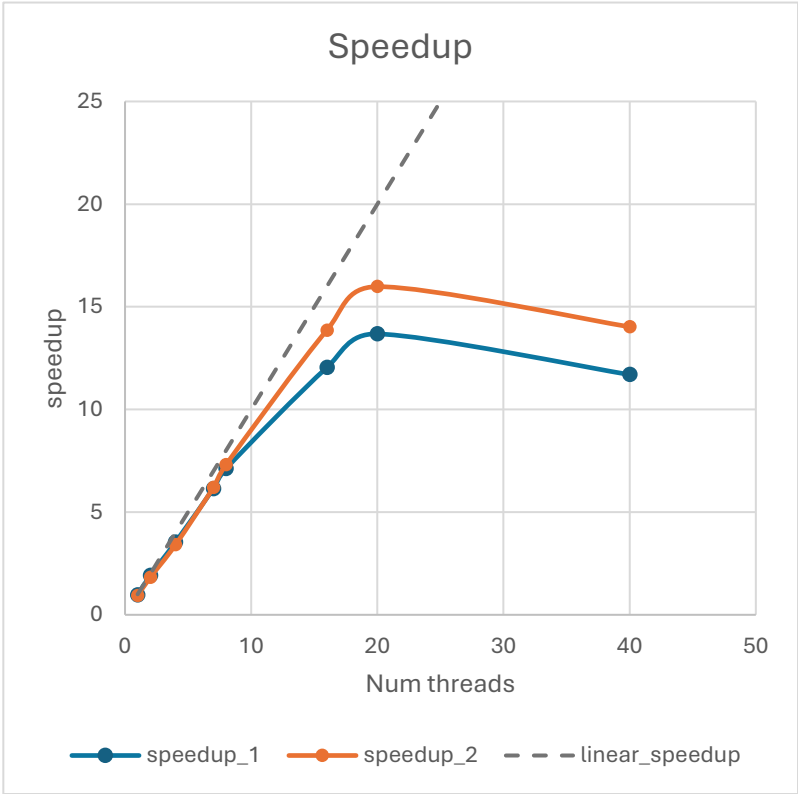


Спецификация

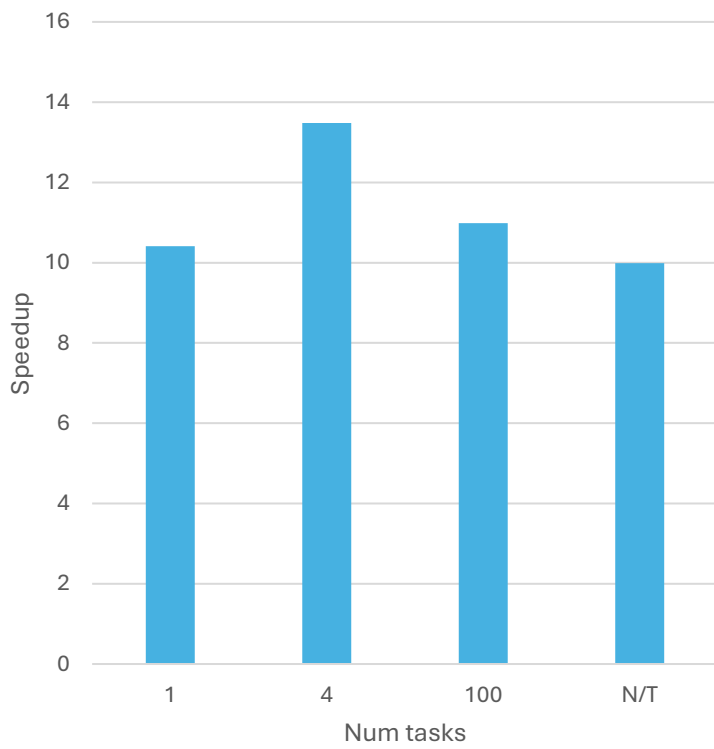
Server name	ProLiant XL270d Gen10
CPU	Intel(R) Xeon(R) Gold 6248 CPU @ 2.50GHz
Physical cores	20 cores
L1d / L1i	1.3 MiB × 40 instances each
L2	40 MiB × 40 instances
L3	55 MiB (2 instances)
NUMA nodes	2
RAM (node0 node1)	385636 MB 387008 MB
OS	Ubuntu 22.04.5 LTS

Результаты

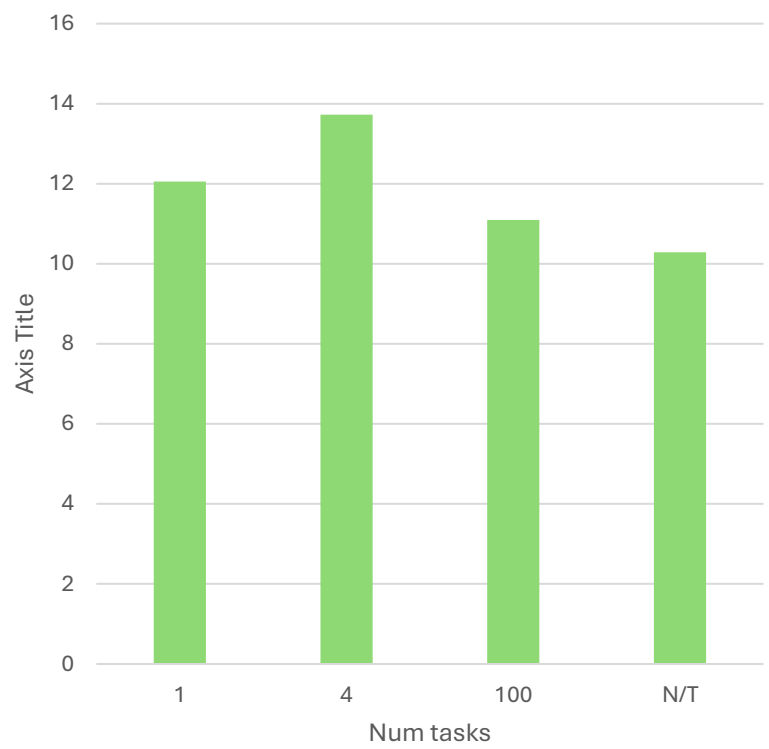
	Thread amount															
	2			4		7		8		16		20		40		
	T1	T2	S2	T4	S4	T7	S7	T8	S8	T16	S16	T20	S20	T40	S40	
prog 1	103.5	55.77	1.913	29.24	3.538	16.63	6.142	14.85	7.117	8.584	12.042	7.512	13.679	8.825	11.694	
prog 2	103.5	58.63	1.820	30.33	3.411	16.49	6.194	14.46	7.310	7.459	13.858	6.430	15.981	7.359	14.023	



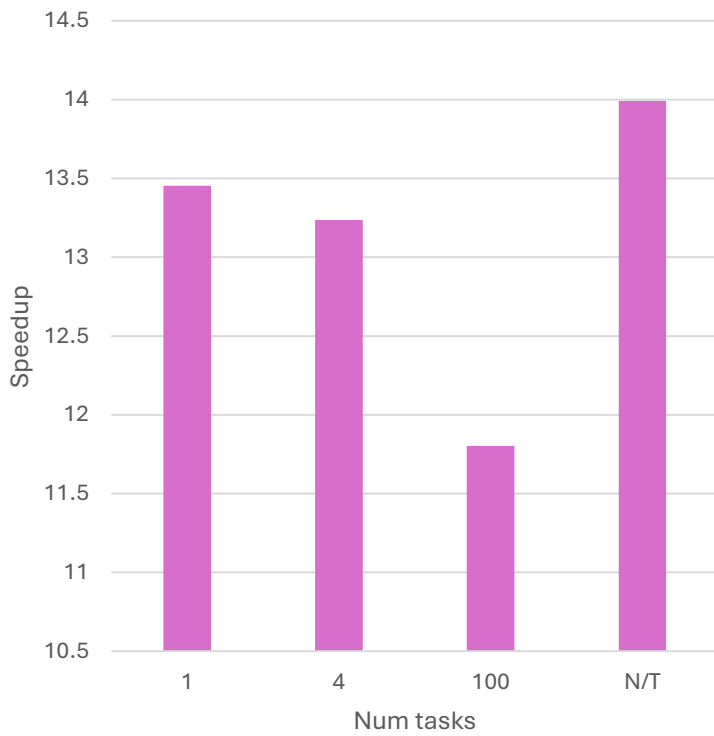
Static



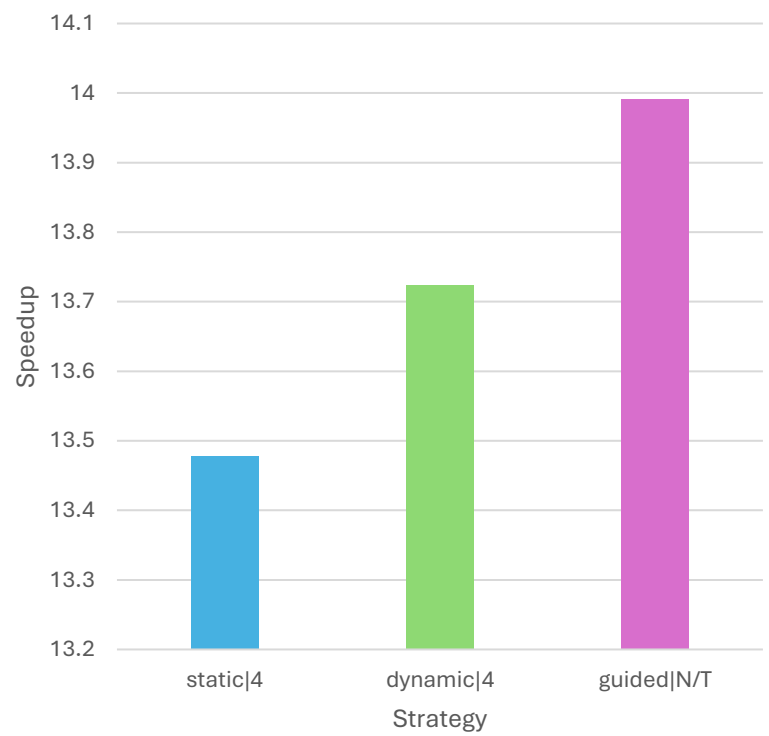
Dynamic



guided



Summary



Выводы

Наиболее линейный прирост дал второй вариант исполнения программы, потому что создание потоков и распределение их по ядрам происходит один раз в начале итерационного процесса, после чего потоки переиспользуются на всех итерациях без повторных накладных расходов на запуск и завершение параллельных секций. Кроме того, уменьшается число барьеров синхронизации между отдельными параллельными участками и упрощается обмен данными между итерациями (данные остаются в кеше, что снижает нагрузку на подсистему памяти), за счёт чего масштабирование с увеличением числа потоков становится более близким к линейному.

Самой оптимальной конфигурацией планировщика оказалась `guided` с количеством задач на поток, равным (N/T) , где (N) – общее количество задач, а (T) – количество потоков, потому что эта схема сначала выдаёт потоку более крупные порции работы, а по мере обработки — всё меньшие, автоматически подстраиваясь под оставшийся объём вычислений. В результате уменьшаются накладные расходы по сравнению с мелкозернистым `dynamic` и одновременно достигается хорошая балансировка нагрузки при неравномерной трудоёмкости отдельных задач, что может снизить простой потоков и улучшить утилизацию вычислительных ресурсов.